

微算機系統 2025
MICROPROCESSOR SYSTEMS

4-stage Pipeline CPU

Group 34

110590042 柳艾德

112590002 林羿安

Content

Instruction	Instruction code			usage	Result
	opcode	Rs	Rt		
Load	0000	2bit	2bit	Load Rs Data	$Rs \leftarrow \text{Data}$
Move	0001			Move Rs, Rt	$Rs \leftarrow Rt$
Add	0010			Add Rs, Rt	$Rs \leftarrow Rs + Rt$
And	0011			And Rs, Rt	$Rs \leftarrow Rs \& Rt$
Sub(A-B)	0101			Sub Rs, Rt	$Rs \leftarrow Rs - Rt$
Sub(B-A)	1001			Sub Rs, Rt	$Rs \leftarrow Rt - Rs$
Nor	0110			Nor Rs, Rt	$Rs \leftarrow Rs \text{ Nor } Rt$
Slt	0100			Slt Rs, Rt	if ($Rs < Rt$) $Rs = 1$; else $Rs = 0$
Div	1000			Div Rs, Rt	$Rs \leftarrow Rs / Rt$

This week's lab focuses on designing a simple 4-stage pipelined CPU capable of executing basic arithmetic and logic operations. The CPU includes a small register file with four 8-bit registers and uses RS and RT fields to select source and destination operands. Instructions are input manually through switches and move through stages for fetching, decoding, execution, and write-back. Forwarding logic is implemented to handle data hazards without stalling, and LEDs are used to show the status of each pipeline stage, while seven-segment displays reflect register values and input data in real time.

Process & Results

處理流程利用 process 同時執行的特性來模擬 pipeline 的結構, 並將 each stage 所需的 register content 用 signal 的方式儲存, 方便資料的傳遞。並利用 valid buffer 來控制 stage 的啟動和關閉。

- **Each Stage**

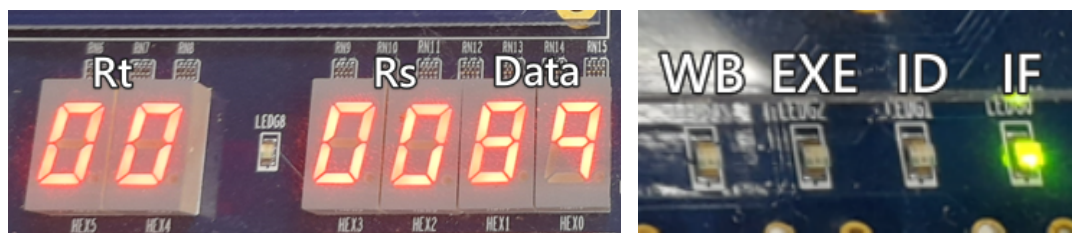
IF: 依照 Lab8 Port Map 讀取指令

ID: 把 Fetch 的 Instruction 所得的 Rs & Rt Address 轉換成 Data

EXE: 將 Data(Rs & Rt) 通過 Opcode 取得 Result

WB: 將 Result 寫回 Rs 所指向的 Register File (燈亮即寫回)

- **Display**



- **PS**

1. Opcode “1111” 為 NOP

2. Div 未實作

3. Hazard 未實作

● Test Data

TEST 1							
Load r0, 10001001 NOP (opcode = '1111') NOP (opcode = '1111') Move r1, r0							
Clk	Data	Opcode	Rs	Rt	Rs_Data	Rt_Data	Result
1	Data	Opcode	Rs	Rt	Rs_Data	Rt_Data	Result
IF	10001001	0000	00	X			
ID	X	X	X	X	X	X	
EXE		X	X		X	X	X
WB			X				X
2	Data	Opcode	Rs	Rt	Rs_Data	Rt_Data	Result
IF	X	X	X	X			
ID	10001001	0000	00	X	10001001	X	
EXE		X	X		X	X	X
WB			X				X
3	Data	Opcode	Rs	Rt	Rs_Data	Rt_Data	Result
IF	X	X	X	X			
ID	X	X	X	X	X	X	
EXE	X	0000	00		10001001	X	10001001
WB			X				X

4	Data	Opcode	Rs	Rt	Rs_Data	Rt_Data	Result
IF	X	0001	01	00			
ID	X	X	X	X	X	X	
EXE		X	X		X	X	X
WB			00				10001001
5	Data	Opcode	Rs	Rt	Rs_Data	Rt_Data	Result
IF	X	X	X	X			
ID	X	0001	01	00	X	10001001	
EXE		X	X		X	X	X
WB			X				X
6	Data	Opcode	Rs	Rt	Rs_Data	Rt_Data	Result
IF	X	X	X	X			
ID	X	X	X	X	X	X	
EXE		0001	01		X	10001001	10001001
WB			X				X
7	Data	Opcode	Rs	Rt	Rs_Data	Rt_Data	Result
IF	X	X	X	X			
ID	X	X	X	X	X	X	
EXE		X	X		X	X	X
WB			01				10001001

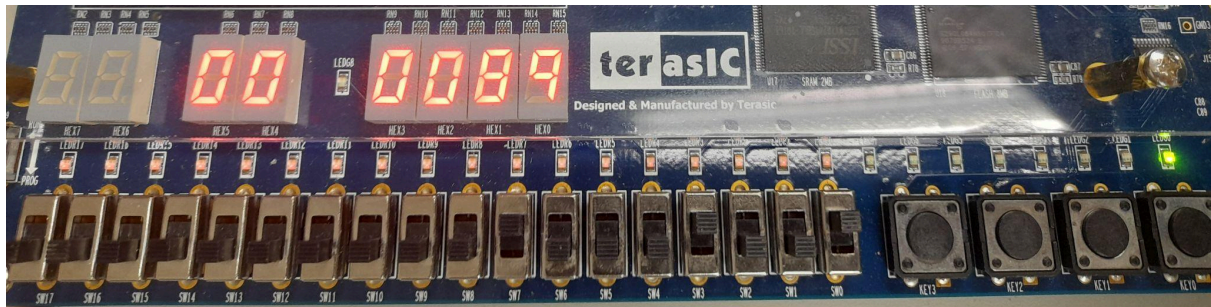
TEST 2							
Load r0, 00001000 NOP NOP add r0, r1							
Clk	Data	Opcode	Rs	Rt	Rs_Data	Rt_Data	Result
1	Data	Opcode	Rs	Rt	Rs_Data	Rt_Data	Result
IF	00001000	0000	00	X			
ID	X	X	X	X	X	X	
EXE		X	X		X	X	X
WB			X				X
2	Data	Opcode	Rs	Rt	Rs_Data	Rt_Data	Result
IF	X	X	X	X			
ID	00001000	0000	00	X	00001000	X	
EXE		X	X		X	X	X
WB			X				X
3	Data	Opcode	Rs	Rt	Rs_Data	Rt_Data	Result
IF	X	X	X	X			
ID	X	X	X	X	X	X	
EXE		0000	00		00001000	X	00001000
WB			X				X

4	Data	Opcode	Rs	Rt	Rs_Data	Rt_Data	Result
IF	X	0010	00	01			
ID	X	X	X	X	X	X	
EXE		X	X		X	X	X
WB			00				00001000
5	Data	Opcode	Rs	Rt	Rs_Data	Rt_Data	Result
IF	X	X	X	X			
ID	X	0010	00	01	00001000	10001001	
EXE		X	X		X	X	X
WB			X				X
6	Data	Opcode	Rs	Rt	Rs_Data	Rt_Data	Result
IF	X	X	X	X			
ID	X	X	X	X	X	X	
EXE		0010	00		00001000	10001001	10010001
WB			X				X
7	Data	Opcode	Rs	Rt	Rs_Data	Rt_Data	Result
IF	X	X	X	X			
ID	X	X	X	X	X	X	
EXE		X	X		X	X	X
WB			00				10010001

2025/06/18

[TEST 1]

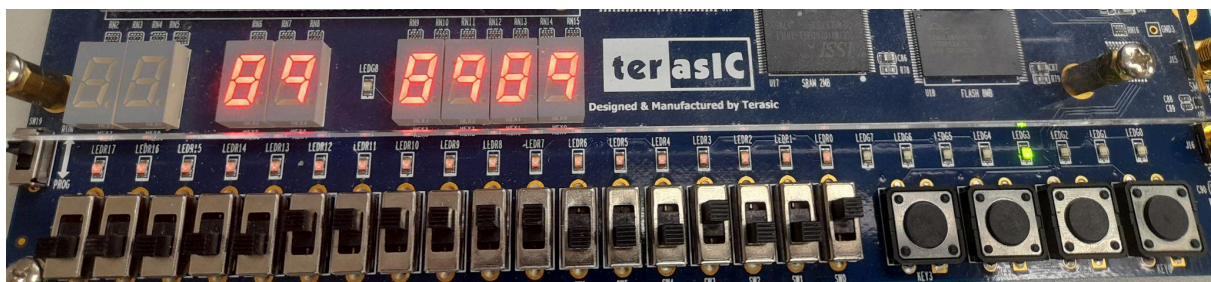
Clk = 1, Load 00, 0x89



Clk = 4, Move 01, 00



Clk = 7



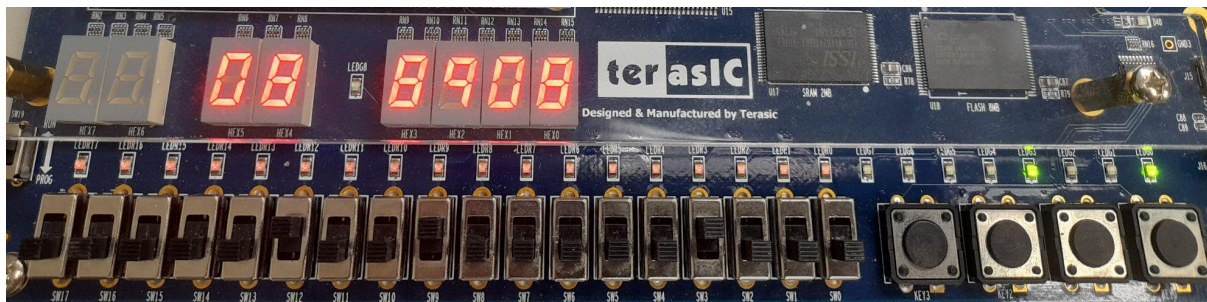
2025/06/18

[TEST 2]

Clk = 1, Load 00, 0x08



Clk = 4, Add 00, 01



Clk = 7, Result = 0x08 + 0x89 = 0x91



Code

```

LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.NUMERIC_STD.ALL;
USE work.MyPackage.ALL;

ENTITY SampleCPU IS
    PORT (
        -- button input
        clk : IN STD_LOGIC;
        -- switches input
        Opcode : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
        Rs : IN STD_LOGIC_VECTOR(1 DOWNTO 0);
        Rt : IN STD_LOGIC_VECTOR(1 DOWNTO 0);
        Data : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
        -- Hex output
        HEX0, HEX1, HEX2, HEX3, HEX4, HEX5 : OUT STD_LOGIC_VECTOR(6 DOWNTO 0);
        -- Stage
        LEDG : out STD_LOGIC_VECTOR(3 DOWNTO 0)
        -- LEDR : out std_logic;
    );
END SampleCPU;

ARCHITECTURE Behavioral OF SampleCPU IS

    type reg_array is array (0 to 3) of STD_LOGIC_VECTOR(7 downto 0);
    signal reg_file : reg_array := (others => (others => '0')); -- r0 ~ r3

    signal valid : std_logic_vector(3 downto 0) := (others => '0');

    -- IF/ID pipeline registers
    signal IF_ID_Data : STD_LOGIC_VECTOR(7 downto 0);
    signal IF_ID_Opcode : STD_LOGIC_VECTOR(3 downto 0);
    signal IF_ID_Rs : STD_LOGIC_VECTOR(1 downto 0);
    signal IF_ID_Rt : STD_LOGIC_VECTOR(1 downto 0);

    -- ID/EXE pipeline registers
    signal ID_EXE_Opcode : STD_LOGIC_VECTOR(3 downto 0);
    signal ID_EXE_Rs : STD_LOGIC_VECTOR(1 downto 0);
    signal ID_EXE_Rt : STD_LOGIC_VECTOR(1 downto 0);
    signal ID_EXE_Rs_Data : STD_LOGIC_VECTOR(7 downto 0);
    signal ID_EXE_Rt_Data : STD_LOGIC_VECTOR(7 downto 0);

    -- EXE/WB pipeline registers
    signal EXE_WB_Rs : STD_LOGIC_VECTOR(1 downto 0);
    signal EXE_WB_Result : STD_LOGIC_VECTOR(7 downto 0);

    signal rs_seg, rt_seg : STD_LOGIC_VECTOR(7 downto 0);
BEGIN

    -- IF
    process(clk)
    begin
        if rising_edge(clk) then

```

```

        if Opcode = "1111" then -- NOP
            valid(0) <= '0';
        else
            valid(0) <= '1';
            IF_ID_Data <= Data;
            IF_ID_Opcode <= Opcode;
            IF_ID_Rs <= Rs;
            IF_ID_Rt <= Rt;
        end if;
    end if;
end process;

-- ID
process(clk)
begin
    if rising_edge(clk) then
        if valid(0) = '0' then
            valid(1) <= '0';
        else
            valid(1) <= '1';
            ID_EXE_Opcode <= IF_ID_Opcode;
            ID_EXE_Rs <= IF_ID_Rs;
            ID_EXE_Rt <= IF_ID_Rt;

            if IF_ID_Opcode = "0000" then -- Load
                ID_EXE_Rs_Data <= IF_ID_Data;
            else
                ID_EXE_Rs_Data <= reg_file(to_integer(unsigned(IF_ID_Rs)));
            end if;
            ID_EXE_Rt_Data <= reg_file(to_integer(unsigned(IF_ID_Rt)));
        end if;
    end if;
end process;

-- EXE
process(clk)
    variable result : std_logic_vector(7 downto 0) := (others => '0');
begin
    if rising_edge(clk) then
        if valid(1) = '0' then
            valid(2) <= '0';
        else
            valid(2) <= '1';

            case ID_EXE_Opcode is
                when "0000" => -- Load rs <= data
                    result := ID_EXE_Rs_Data;
                when "0001" => -- Move rs <= rt
                    result := ID_EXE_Rt_Data;
                when "0010" => -- Add
                    result := std_logic_vector(unsigned(ID_EXE_Rs_Data) +
unsigned(ID_EXE_Rt_Data));
                when "0011" => -- And
                    result := std_logic_vector(unsigned(ID_EXE_Rs_Data) and
unsigned(ID_EXE_Rt_Data));
                when "0101" => -- Sub: rs - rt

```

```

        result := std_logic_vector(unsigned(ID_EXE_Rs_Data) -
unsigned(ID_EXE_Rt_Data));
        when "1001" => -- Sub: rt - rs
            result := std_logic_vector(unsigned(ID_EXE_Rt_Data) -
unsigned(ID_EXE_Rs_Data));
        when "0110" => -- Nor
            result := std_logic_vector(not (unsigned(ID_EXE_Rs_Data) or
unsigned(ID_EXE_Rt_Data)));
        when "0100" => -- Slt
            if unsigned(ID_EXE_Rs_Data) < unsigned(ID_EXE_Rt_Data) then
                result := "00000001";
            else
                result := "00000000";
            end if;
        when "1000" => -- Div
            -- TODO:
        when others =>
            result := (others => '0');
        end case;

        EXE_WB_Rs <= ID_EXE_Rs;
        EXE_WB_Result <= result;
    end if;
end if;
end process;

-- WB
process(clk)
begin
    if rising_edge(clk) then
        if valid(2) = '0' then
            valid(3) <= '0';
        else
            valid(3) <= '1';
            reg_file(to_integer(unsigned(EXE_WB_Rs))) <= EXE_WB_Result;
        end if;
    end if;
end process;

-- Output
LEDG <= valid; -- Show the stage of the CPU

rs_seg <= reg_file(to_integer(unsigned(Rs)));
rt_seg <= reg_file(to_integer(unsigned(Rt)));

BusHex0: HexDisplay port map(Hex => Data(3 downto 0), Seg => HEX0);
BusHex1: HexDisplay port map(Hex => Data(7 downto 4), Seg => HEX1);
RsHex0: HexDisplay port map(Hex => rs_seg(3 downto 0), Seg => HEX2);
RsHex1: HexDisplay port map(Hex => rs_seg(7 downto 4), Seg => HEX3);
RtHex0: HexDisplay port map(Hex => rt_seg(3 downto 0), Seg => HEX4);
RtHex1: HexDisplay port map(Hex => rt_seg(7 downto 4), Seg => HEX5);
END Behavioral;

```